SCI-OCR

PDF to Markdown Pipeline Specification

Current architecture, artifact contracts, runtime services, limitations, and roadmap

Document type:	Current technical specification
Primary goal:	Convert PDFs into LLM-ready Markdown with structured debug artifacts
Main input:	PDF document
Main output:	`output/article.md` plus normalized JSON artifacts
Runtime model:	FastAPI orchestrator plus replaceable HTTP worker services
Current real backends:	PP-DocLayoutV3 CPU, GLM-OCR CPU, optional llama-server vision adapter
Updated:	April 28, 2026

Contents

1. Executive Summary

SCI-OCR is a local-first document processing pipeline that receives a PDF and produces an ordered Markdown article, together with a complete set of intermediate artifacts for inspection and debugging.

The current implemented pipeline is:

```
Input PDF
  -> Page Rendering
  -> Layout
  -> Layout Assets and Routing
  -> OCR + optional Vision
  -> Assembly
  -> output/article.md
```

The system is no longer only a placeholder scaffold. It has a working FastAPI orchestrator, stable Pydantic service contracts, a real CPU PP-DocLayoutV3 layout service, a real GLM-OCR worker, an optional llama-server-backed vision adapter, deterministic assembly, representative PDF fixtures, and a job report script.

> **Note.** The core engineering principle remains unchanged: model backends must be replaceable without changing downstream artifact contracts.

2. Current Capabilities

The current repository supports:

- FastAPI orchestrator with `GET /health` and `POST /run`.
- Input file validation and unique job creation.
- Persistent job workspace under `jobs/output/<job_id>/`.
- `meta.json`, `trace.json`, and `logs.jsonl`.
- PDF page rendering to PNG at 300 DPI by default, with optional 400 DPI.
- HTTP layout service calls for rendered pages.
- Layout stub service for fast contract tests.
- CPU PP-DocLayoutV3 service preserving native labels as `layout_label`.
- Raw and normalized layout artifacts.
- Layout overlays and block crops for every detected layout block.
- Deterministic routing from layout labels to OCR or vision.
- Shared OCR contract and OCR stub.
- GLM-OCR worker behind the same OCR contract.
- Optional vision contract and `vision_llama` adapter.
- Pending vision manifest when the vision backend is unavailable.
- Deterministic assembly into `debug/content_stream.json` and `output/article.md`.
- Representative fixtures for text, tables, formulas, images, bar charts, and line charts.
- `scripts/report_job.py` for manual validation of persisted job artifacts.

- Tests covering contracts, routing, assembly, fixtures, service stubs, and selected integrations.

3. Repository Structure

```
sci-ocr/
+-- orchestrator/
|   +-- app/
|       +-- main.py
|       +-- pipeline.py
|       +-- preparing_for_layout.py
|       +-- layout_stage.py
|       +-- layout_assets_stage.py
|       +-- block_routing.py
|       +-- ocr_stage.py
|       +-- vision_stage.py
|       +-- assembly_stage.py
|       +-- schemas.py
|       +-- config.py
+-- services/
|   +-- layout_stub/
|   +-- layout_ppdoclayoutv3_cpu/
|   +-- ocr_stub/
|   +-- ocr_glm/
|   +-- vision_llama/
|   +-- assembly_stub/
+-- shared/
|   +-- contracts/
|       +-- layout.py
|       +-- ocr.py
|       +-- vision.py
+-- jobs/
|   +-- input/
|   +-- output/
+-- models/
+-- local_tools/
+-- docs/
+-- scripts/
+-- tests/
+-- docker-compose.yml
+-- README.md
```

4. Pipeline Flow

4.1 API Request

The orchestrator exposes:

```
GET /health
POST /run
```

The `/run` request body is:

```
{
  "input_path": "C:/input/test.pdf",
  "dpi": 300
}
```

`dpi` is optional and can be 300 or 400.

4.2 Execution Order

```
validate input
  -> create job_id
  -> create job folders
  -> copy original file
  -> write metadata, trace, log
  -> render PDF pages
  -> call layout service
  -> write raw and normalized layout artifacts
  -> create overlays and crops
  -> route crops to OCR or vision
  -> call OCR service for text/table/formula crops
  -> call optional vision service for image/chart crops
  -> write pending vision manifest if needed
  -> build content_stream.json
  -> render output/article.md
```

5. Job Artifact Contract

Every run creates:

```
jobs/output/<job_id>/
+-- original/
+-- preprocessed/
+-- assets/
|   +-- pages/
|   +-- crops/
|   +-- layout/
+-- debug/
|   +-- preparing_for_layout.json
|   +-- layout_raw_page_0001.json
|   +-- layout_normalized_page_0001.json
|   +-- layout_assets.json
|   +-- ocr_manifest.json
|   +-- ocr_raw_page_0001.json
|   +-- ocr_normalized_page_0001.json
|   +-- vision_manifest.json
|   +-- vision_raw_page_0001.json
|   +-- vision_normalized_page_0001.json
|   +-- vision_pending_manifest.json
|   +-- content_stream.json
|   +-- assembly_manifest.json
+-- output/
|   +-- article.md
+-- meta.json
+-- trace.json
+-- logs.jsonl
```

5.1 Artifact Meaning

- `original/`: unchanged input file.
- `assets/pages/`: rendered page PNG files.
- `assets/layout/`: human-readable layout overlays.
- `assets/crops/`: crops for every layout block.

- `debug/`: normalized and raw stage artifacts.
- `output/article.md`: final LLM-ready article representation.
- `meta.json`: high-level job and stage summary.
- `trace.json`: ordered event timeline.
- `logs.jsonl`: append-only structured logs.

6. Layout Service

6.1 Role

Layout is an external persistent HTTP service. It receives one rendered page image and returns structured layout blocks. The layout service owns detection and block typing. It does not run OCR, generate crops, or assemble Markdown.

Current layout implementations:

- `services/layout_stub`: fast deterministic stub.
- `services/layout_ppdoclayoutv3_cpu`: CPU PP-DocLayoutV3 backend.

6.2 Endpoints

```
GET  /health
GET  /ready
POST /layout
```

6.3 Normalized Layout

The service response is persisted as raw output, then normalized into a stable format used by downstream stages:

```
{
  "stage": "layout",
  "status": "completed",
  "source": "PP-DocLayoutV3",
  "job_id": "job-0001",
  "page_number": 1,
  "image": {
    "path": ".../assets/pages/page_0001.png",
    "width": 2480,
    "height": 3508
  },
  "blocks": [
    {
      "block_id": "p1_b1",
      "type": "text",
      "layout_label": "content",
      "bbox": [100, 100, 700, 180],
      "confidence": 0.98,
      "order": 1,
      "source": "PP-DocLayoutV3"
    }
  ],
  "warnings": []
}
```

7. Block Routing

Routing is deterministic and model-free. It maps layout labels to downstream services, recognition tasks, requested output formats, and assembly roles.

```
text-like blocks -> OCR    -> markdown
table blocks     -> OCR    -> markdown
formula blocks   -> OCR    -> latex
image/chart      -> Vision -> markdown or pending placeholder
```

Native PP-DocLayoutV3 labels are preferred when available because they preserve more semantic detail than canonical block types.

7.1 Important Label Families

Label family	Target	Task	Format
Text-like labels	OCR	text	markdown
Table	OCR	table	markdown
Display/inline formulas	OCR	formula	latex
Image labels	Vision	image	none
Chart labels	Vision	chart	none

Images and charts are intentionally not sent to OCR. They require visual description, chart extraction, diagram reconstruction, or placeholder handling.

8. OCR Service

8.1 Role

OCR is an external persistent worker service. It receives one cropped block image and returns recognized content in the format requested by the orchestrator.

Current OCR implementations:

- `services/ocr_stub`: deterministic contract-compatible stub.
- `services/ocr_glm`: real GLM-OCR worker.

Docker Compose points OCR traffic at `ocr_glm` by default.

8.2 Endpoints

```
GET  /health
GET  /ready
POST /ocr
```

8.3 GLM-OCR Prompt Mapping

```
text    -> Text Recognition:
table   -> Table Recognition:
formula -> Formula Recognition:
```

Formula responses are normalized by removing outer display wrappers such as `$$...$$`. Assembly owns final Markdown wrapping.

9. Vision Service

9.1 Role

Vision is optional. It receives image/chart/figure crops and returns Markdown-ready visual content. If the service is not configured, unavailable, or times out, the pipeline writes a pending vision manifest and continues:

```
debug/vision_pending_manifest.json
```

Current implementation:

- `services/vision_llama`: adapter around an external multimodal `llama-server`.

9.2 Runtime Split

```
Windows host:
  llama-server on http://127.0.0.1:8080

Docker:
  vision_llama service on http://vision_llama:8000
  calls http://host.docker.internal:8080
```

9.3 Vision Output

The adapter asks the model to classify a visual crop as:

```
photo_or_illustration
chart_or_plot
diagram_or_flowchart
table_like_visual
unknown
```

It returns Markdown for descriptions, charts, approximate data tables, or Mermaid diagrams when possible.

10. Assembly

Assembly is currently an orchestrator module rather than a separate worker. It does not perform model inference. It reads normalized layout, OCR, vision, and pending vision artifacts.

It produces:

```
debug/content_stream.json
debug/assembly_manifest.json
output/article.md
```

The current ordering rule is:

```
page_number -> order -> bbox top -> bbox left -> block_id
```

Rendering behavior:

- titles become Markdown `#` headings.
- headings become Markdown `##` headings.
- text remains Markdown paragraphs.

- tables are inserted as Markdown or HTML returned by OCR.
- formulas are wrapped as display or inline LaTeX.
- completed visual blocks insert vision Markdown.
- pending visual blocks become blockquote placeholders.

11. Docker Compose Runtime

Docker Compose currently includes:

- `orchestrator`
- `layout_stub`
- `layout_ppdoclayoutv3_cpu`
- `ocr_stub`
- `ocr_glm`
- `vision_llama`
- `assembly_stub`

The default real pipeline path is:

```
orchestrator
  -> layout_ppdoclayoutv3_cpu
  -> ocr_glm
  -> vision_llama, if external llama-server is ready
```

Important timeouts:

```
LAYOUT_TIMEOUT_SECONDS=120
OCR_TIMEOUT_SECONDS=600
VISION_TIMEOUT_SECONDS=1200
```

The OCR image installs CPU PyTorch wheels explicitly:

```
torch==2.9.1+cpu
torchvision==0.24.1+cpu
```

12. Representative Fixtures and Validation

Fixtures can be regenerated with:

```
.\.venv\Scripts\python.exe scripts\generate_test_pdfs.py
```

Important fixture PDFs:

- `formula_table_fixture.pdf`: compact one-page smoke document.
- `science_mixed_content.pdf`: two-page document with headings, prose, a table, formulas, embedded image, bar chart, and line chart.

After a run, inspect a job with:

```
.\.venv\Scripts\python.exe scripts\report_job.py <job_id>
```

The report summarizes stage statuses, layout labels, routing split, OCR formats, vision status, assembly sources, and artifact paths.

13. Known Verified Run

A full local run on `science_mixed_content.pdf` completed with:

```
layout:   completed, 18 blocks, source=PP-DocLayoutV3
ocr:      completed, 15 blocks, source=GLM-OCR
vision:   completed, 3 blocks, source=qwen3.6-27b-q4_k_m
assembly: completed, 18 blocks
pending vision blocks: 0
```

The assembled article included real text, table output, LaTeX formulas, and vision-generated chart descriptions.

14. CPU Layout Scaling Notes

PP-DocLayoutV3 through PaddleOCR/Paddle already uses internal CPU parallelism. Current installed defaults are:

```
DEFAULT_ENABLE_MKLDNN = True
DEFAULT_CPU_THREADS = 10
DEFAULT_MKLDNN_CACHE_CAPACITY = 10
```

Local measurements:

```
1 unrestricted layout worker:
  about 5.3 seconds per page

3 separate layout containers limited to 2 CPU each:
  about 18.8 seconds per page on average
```

The second setup was slower because each worker was starved. One Paddle inference process wants roughly 8-10 CPU threads.

Memory estimate:

```
observed warmed worker peak: about 668 MiB
planning estimate: about 1.0 GiB
conservative budget: about 1.2 GiB per worker
```

Future high-core machines should scale layout with a worker pool:

```
worker_count = floor(usable_cpu_cores / 10)
also cap by available_memory_gb / 1.2
```

Example sizing:

```
8 cores   -> 1 worker
10 cores  -> 1 worker
15 cores  -> 1 worker
20 cores  -> 2 workers
32 cores  -> 3 workers
64 cores  -> 6 workers
400 cores -> about 40 workers, if memory and I/O allow it
```

This is a future development direction, not current implementation. The detailed planning note lives in `docs/LAYOUT_SCALING.md`.

15. Current Limitations

- PP-DocLayoutV3 is wired as a CPU-only service and still needs broader quality validation on representative documents.
- GLM-OCR CPU inference is slow because crops are processed one at a time.
- `vision_llama` depends on an external multimodal `llama-server`; on CPU-class hardware it can be slow.
- Chart extraction from vision output is approximate.
- Assembly does not yet repair multi-column reading order, merge broken paragraphs, or associate captions with figures/charts beyond deterministic layout order.
- There is not yet a GPU PP-DocLayoutV3 container.
- There is not yet a quality validation harness with expected semantic assertions for representative jobs.

16. Roadmap

16.1 Near Term

- Add a quality validation harness for representative job artifacts.
- Validate CPU PP-DocLayoutV3 output quality on real and synthetic PDFs.
- Validate GLM-OCR output quality on text, table, and formula crops.
- Validate `vision_llama` output on images, charts, and diagrams.
- Improve vision prompts and normalization for chart data and Mermaid.

16.2 Performance

- Improve OCR throughput with batching, crop parallelism, or GPU execution.
- Add future layout worker-pool support for high-core machines.
- Keep one-worker `LAYOUT_SERVICE_URL` mode for ordinary desktops.
- Consider `LAYOUT_SERVICE_URLS` for multi-worker page distribution.

16.3 Backends

- Add a separate GPU PP-DocLayoutV3 service/container.
- Replace or augment `vision_llama` with specialized image, chart, and diagram processors when quality requirements are clearer.
- Keep all replacements behind stable shared contracts.

16.4 Assembly Quality

- Improve multi-column reading order.
- Merge paragraphs split across layout blocks.
- Associate captions with figures and charts.
- Optionally filter headers, footers, and page numbers.

17. Final Summary

SCI-OCR is now best understood as an artifact-driven, replaceable-backend pipeline:

```
FastAPI orchestrator
  + stable shared contracts
  + PP-DocLayoutV3 CPU layout
  + GLM-OCR CPU OCR
  + optional llama-server vision
  + deterministic assembly
  + persistent artifacts for every stage
```

The next important engineering move is not another placeholder service. It is quality measurement: build validation around representative PDFs so layout, OCR, vision, and assembly improvements can be judged with repeatable evidence.